Project Proposal Template

Project: E-Commerce Data Integration & Analytics Pipeline

Objective: To build an end-to-end e-commerce data analytics platform that keeps sales transactions in an operational MySQL database and a scraped product catalogue in MongoDB, moves new data incrementally into a PostgreSQL star-schema data warehouse with Python ETL orchestrated by Apache Airflow, and reports it with SQL analytics and a Power BI dashboard.

Description: An e-commerce company records its daily sales in an operational (OLTP) database and collects a product catalogue from a retail website, but management has no single place for reporting. This project builds that platform in five modules that follow the structure of the reference repository (OLTP, NoSQL, data warehouse, analytics and dashboard, ETL and pipelines), using real data instead of synthetic data: the Olist Brazilian e-commerce dataset (about 100,000 orders, 2016–2018), loaded into MySQL in monthly batches to simulate new sales, and the product catalogue of The Conran Shop Kuwait (about 1,900 products), collected with a web scraper that follows the site's robots.txt (collection pages only, one request per second). Amounts are kept in their source currency: BRL for Olist sales and KWD for catalogue prices.

Tools & Design: The platform has five modules, each delivered as a numbered folder in the GitHub repository with its scripts, data samples, screenshots and a Markdown guide, as shown below:

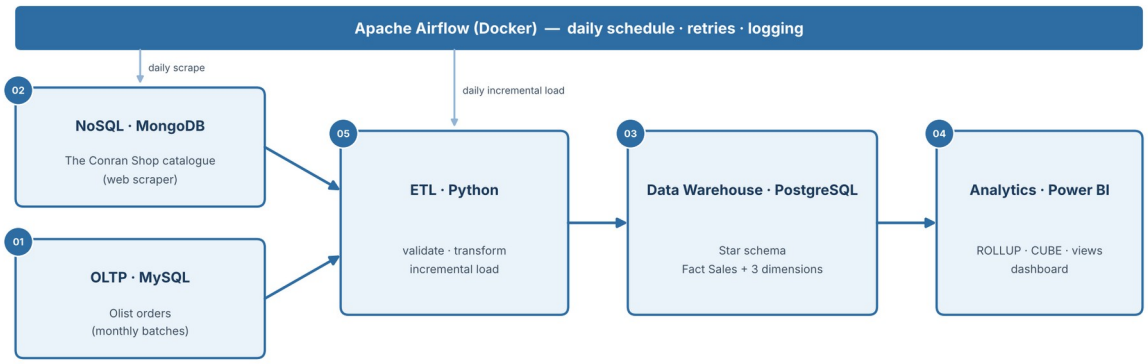


Figure 1. Architecture: two operational sources (MySQL, MongoDB), Python ETL orchestrated by Apache Airflow, a PostgreSQL data warehouse and a Power BI dashboard.

- 01 OLTP (MySQL) — Olist customers, sellers, products, orders, order items, payments and reviews in a normalised schema; index on the purchase timestamp; export script with mysqldump.
- 02 NoSQL (MongoDB) — the scraped catalogue stored as documents; indexes, queries and export to CSV.

- 03 Data Warehouse (PostgreSQL) — star schema with Fact Sales, Dim Customer, Dim Product and Dim Date, designed for four company reports.
- 04 Analytics & Dashboard — GROUPING SETS, ROLLUP, CUBE and a materialized view in SQL, and a Power BI dashboard.
- 05 ETL & Pipelines — Python incremental ETL from MySQL and MongoDB to PostgreSQL with data validation, run by Apache Airflow DAGs with schedules, retries and logging.

Technologies to Use: The following technologies are used across the platform:

- Python (pandas, requests, BeautifulSoup, mysql-connector-python, psycopg2, pymongo) — scraper and ETL scripts
- MySQL — operational (OLTP) sales database
- MongoDB — NoSQL product catalogue
- PostgreSQL — data warehouse (star schema)
- Apache Airflow — orchestration, scheduling, retries and logging
- Docker Compose — runs Airflow and the three databases
- Power BI Desktop — business-intelligence dashboard
- DBeaver — database management and querying (MySQL and PostgreSQL)
- Bash and Git / GitHub — export scripts, version control and project repository

---

Team Members & Responsibilities:

1- Mahmoud Dahroug — Team Leader & BI — Coordination, data dictionary, Power BI dashboard, README and architecture diagram, final presentation

2- Aya Elgarwany — Data Extraction & NoSQL — Web scraper, MongoDB catalogue, Olist batch files, transformation functions

3- Hussein Saeed — Database Engineer — MySQL OLTP database, PostgreSQL star schema and ERD, analytical SQL

4- Nouran Elshwehy — ETL & Orchestration — Docker setup, incremental Python ETL, data validation, Airflow DAGs

5- Islam Hassan — Documentation & Testing Support — Module guides, screenshots, test runs, proofreading

---

Week 1: Operational Databases & Data Collection

- Mahmoud — Create the GitHub repository with the module folders (01_oltp to 05_etl) and the README skeleton, and update the data dictionary to the new design.
- Nouran — Set up Docker Compose with Airflow, MySQL, MongoDB and PostgreSQL, and run a first test DAG.
- Hussein — Design the MySQL OLTP schema for the Olist data (customers, sellers, products, orders, order items, payments, reviews), write oltp.sql, load the first monthly batches and add an index on the purchase timestamp.
- Hussein — Write datadump.sh to export the OLTP database with mysqldump.
- Aya — Adapt the scraper to The Conran Shop collection pages (?page=N, one request per second, clear User-Agent): name, brand, product type, sale and original price, discount, stock status and URL, saved as dated JSON.
- Aya — Import the scraped catalogue into catalog.products in the MongoDB container with mongoimport.
- Islam — Prepare the Markdown template for the module guides and capture the Module 1 screenshots (tables, row counts, index).

Deliverables:

- Repository structure, README skeleton and updated data dictionary
- Docker Compose with Airflow running a test DAG
- MySQL OLTP schema and loaded data (oltp.sql)
- Export script (datadump.sh) and database dump
- Conran Shop scraper and dated JSON snapshot
- MongoDB catalogue collection

Week 2: Data Warehouse, ETL & NoSQL Queries

- Hussein — Update the PostgreSQL star schema (Fact Sales, Dim Customer, Dim Product, Dim Date), write schema.sql, generate Dim Date and redraw the ERD.

- Aya — Write the transformation functions used by the ETL: data types, missing values, duplicates, category translation and catalogue price parsing.
- Nouran — Write the Python connection modules (mysqlconnect.py, postgresqlconnect.py) and the incremental ETL (automation.py) that loads only new orders using a last-loaded marker.
- Aya — Create MongoDB indexes, write the catalogue queries (counts by brand and product type, average price, stock status) and export to CSV with mongoexport.
- Mahmoud — Install Power BI Desktop, connect it to the PostgreSQL warehouse, and review the Week 1 deliverables.
- Islam — Capture the Module 2 and 3 screenshots and record the results of the first ETL test runs.

Deliverables:

- Data-warehouse schema (schema.sql) and ERD
- Python incremental ETL scripts
- MongoDB indexes, queries and CSV export
- Draft module guides for Modules 1–3 with screenshots

Week 3: Analytics, Dashboard & Orchestration

- Hussein — Write the analytical SQL: GROUPING SETS, ROLLUP, CUBE, a materialized view, and the four company reports (sales by year and state, month and category, quarter and state, category and state).
- Nouran — Build the Airflow DAG for the daily incremental load (extract new orders → validate → transform → load → quality check) with logging and retries.
- Aya — Build the Airflow DAG for the daily catalogue scrape (scrape → MongoDB → Dim Product).
- Mahmoud — Build the Power BI dashboard: four report pages and a summary page.
- Islam — Capture the Airflow screenshots (DAG list, unpause, DAG runs, graph view, logs) and the Power BI screenshots.

Deliverables:

- Analytical SQL scripts and materialized view
- Airflow DAGs with successful runs
- Power BI dashboard (.pbix file and screenshots)
- Airflow and dashboard screenshots

Week 4: Testing, Documentation & Final Presentation

- Mahmoud — Coordinate an end-to-end run with a new monthly batch (MySQL → Airflow → PostgreSQL → Power BI).
- Hussein & Nouran — Run failure tests (bad batch, database down, website unreachable) to show retries and error handling, and fix issues.
- Aya, Hussein & Nouran — Finish the Markdown guide of each module (steps, commands, outputs, screenshots) for the modules they own.
- Mahmoud — Finalise the README (architecture diagram, repository structure, how to run) and the final presentation.
- Islam — Proofread all module guides and the README, and check every link and screenshot.
- All team members — Conduct a full dry run of the live demo.

Deliverables:

- Complete GitHub repository following the reference structure
- README with architecture diagram
- Module guides 01–05 (Markdown with screenshots)
- Final presentation and live demo